

Spring Boot-JPA Example

demo2-boot-jpa: Files required

1. MyBootApplication.java	Same as Demo1
2. Customer.java	Same as Demo1
3. CustomerDAO.java	Same as Demo1
4. CustomerDAOImpl.java	Updated in Demo1
5. JLCAppConfig.java	Updated in Demo1
6. application.properties	Same as Demo1
7. pom.xml	Same as Demo1

4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @PersistenceContext
    EntityManager entityManager;

    @Override
    public void addCustomer(Customer cust) {
        entityManager.persist(cust);
    }

    @Override
    public List<Customer> getAllCustomers() {
        String jpaQL= "from Customer cust";
        List<Customer> mylist = entityManager.createQuery(jpaQL, Customer.class).getResultList();
        return mylist;
    }
}
```



```
@Override
public List<Customer> getCustomersByCity(String city) {
String jpQL= "from Customer cust where cust.city=?1";
List<Customer> mylist =
entityManager.createQuery(jpQL, Customer.class)
.setParameter(1, city)
.getResultList();

return mylist;
}
}
```

6)JLAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JLAppConfig {

}
```



Spring JPA

EntityManager IMP Methods

1.	<code>void persist(Object entity)</code>
2.	<code><T> T merge(T entity)</code>
3.	<code>void remove(Object entity)</code>
4.	<code><T> T getReference(Class<T> entityClass, Object primaryKey)</code>
5.	<code><T> T find(Class<T> entityClass, Object PK)</code>
6.	<code><T> T find(Class<T> entityClass, Object PK, LockModeType lockMode)</code>
7.	<code>Query createQuery(String qlStr)</code>
8.	<code><T> TypedQuery<T> createQuery(String qlStr, Class<T> resultClass)</code>
9.	<code>Query createNamedQuery(String name)</code>
10.	<code><T> TypedQuery<T> createNamedQuery(String nm, Class<T> resCls)</code>
11.	<code>Query createNativeQuery(String qlStr)</code>
12.	<code>Query createNativeQuery(String qlStr, Class resultClass)</code>
13.	<code><T> TypedQuery<T> createQuery(CriteriaQuery<T> criteriaQuery)</code>
14.	<code>void refresh(Object entity)</code>
15.	<code>boolean contains(Object entity)</code>
16.	<code>EntityTransaction getTransaction()</code>
17.	<code>void detach(Object entity)</code>
18.	<code>void clear()</code>
19.	<code>void flush()</code>
20.	<code>void close()</code>



Spring Boot-JPA Complete Example

demo3-boot-jpa: Files required

1. MyBootApplication.java	Updated in demo3
2. Customer.java	Same as demo2
3. CustomerDAO.java	Updated in demo3
4. CustomerDAOImpl.java	Updated in demo3
5. application.properties	Updated in demo3
6. pom.xml	Same as demo2

1)MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust1 = new Customer("vas", "vas@jlc", 12345, "Blore");
        custDAO.addCustomer(cust1);

        // 2.Update Customer
        Customer cust2 = new Customer(8,"SD", "SD@jlc", 999, "Delhi");
        custDAO.updateCustomer(cust2);

        //3. Get Customer By Cid
        Customer cust3 =custDAO.getCustomerByCid(8);
        System.out.println("custt 3: "+cust3);
    }
}
```



```
Customer cust4 = custDAO.getCustomerByCid(88);  
System.out.println("custt 4: "+cust4);
```

//4. Delete Customer By Cid

```
custDAO.deleteCustomer(1);  
custDAO.deleteCustomer(101);
```

//5.All Customers - JPAQL

```
List<Customer> mylist1 = custDAO.getAllCustomers();  
for(Customer mycust: mylist1) {  
    System.out.println(mycust);  
}
```

//6.All Customers – Native Query

```
List<Customer> mylist2 = custDAO.getAllCustomers1();  
for(Customer mycust: mylist2) {  
    System.out.println(mycust);  
}
```

//7. Customers By City

```
List<Customer> mylist3 = custDAO.getCustomersByCity("Hyd");  
for(Customer mycust: mylist3) {  
    System.out.println(mycust);  
}
```

//8. Customers By Phone

```
List<Customer> mylist4 = custDAO.getCustomersByPhone(12345);  
for(Customer mycust: mylist4) {  
    System.out.println(mycust);  
}
```

//9. Customers By Email

```
Customer cust5 = custDAO.getCustomerByEmail("hello@jlc");  
System.out.println(cust5);
```

//10. Customers By Email and Phone

```
List<Customer> mylist5= custDAO.getCustomersByEmailAndPhone("hello@jlc", 12345);  
for(Customer mycust: mylist5) {  
    System.out.println(mycust);  
}
```



//10. Customers By Email and Phone

```
List<Customer> mylist6 = custDAO.getCustomersByEmailAndPhone("hai@jlc", 12345);  
for(Customer mycust: mylist6) {  
    System.out.println(mycust);  
}
```

//11. Customers By Email Or Phone

```
List<Customer>mylist7 = custDAO.getCustomersByEmailOrPhone("hai@jlc", 12345);  
for(Customer mycust: mylist7) {  
    System.out.println(mycust);  
}
```

//12. Customers By Email Or Phone

```
List<Customer> mylist8 = custDAO.getCustomers("Blore", "hello@jlc", 12345);  
for(Customer mycust: mylist8 ) {  
    System.out.println(mycust);  
}
```

//13. Customers By NameContains

```
List<Customer> mylist9= custDAO.getCustomersByNameContains("sri");  
for(Customer mycust: mylist9) {  
    System.out.println(mycust);  
}
```

//13. Customers By NameContains

```
List<Customer> mylist10 = custDAO.getCustomersByNameContains("vas");  
for(Customer mycust: mylist10 ) {  
    System.out.println(mycust);  
}
```

//14. Customers Count

```
System.out.println(custDAO.getCustomerCount());
```

//15. Customers Count By City

```
System.out.println(custDAO.getCustomerCountByCity("Blore"));  
System.out.println(custDAO.getCustomerCountByCity("Hyd"));
```

//16. Phone By Email

```
System.out.println(custDAO.getCustomerPhoneByEmail("hello@jlc"));
```

//17. City By Email

```
System.out.println(custDAO.getCustomerCityByEmail("hello@jlc"));
```



//18. Email and Phones of Customers By city

```
Map<String,Integer> mymap= custDAO.getCustomersInfo("Blore");
System.out.println(mymap);
System.out.println("Done!!!");
}

public static void main(String[] args) {
SpringApplication.run(MyBootApplication.class, args);
}
}
```

3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.math.BigInteger;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {

    public void addCustomer(Customer cust);
    public void updateCustomer(Customer cust);
    public void deleteCustomer(int cid);
    public Customer getCustomerByCid(int cid);

    public List<Customer> getAllCustomers();
    public List<Customer> getAllCustomers1();
    public List<Customer> getCustomersByCity(String city);
    public List<Customer> getCustomersByPhone(int phone);

    public Customer getCustomerByEmail(String email);

    public List<Customer> getCustomersByEmailAndPhone(String email,int phone);
    public List<Customer> getCustomersByEmailOrPhone(String email,int phone);
    public List<Customer> getCustomers(String city,String email,int phone);
    public List<Customer> getCustomersByNameContains(String name);

    public BigInteger getCustomerCount();
    public BigInteger getCustomerCountByCity(String city);
    public int getCustomerPhoneByEmail(String email);
    public String getCustomerCityByEmail(String email);
    public Map<String,Integer> getCustomersInfo(String city);
}
```



4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.math.BigInteger;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.Query;

import org.hibernate.dialect.Database;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @PersistenceContext
    EntityManager entityManager;

    @Override
    public void addCustomer(Customer cust) {
        entityManager.persist(cust);
    }

    @Override
    public void updateCustomer(Customer cust) {
        entityManager.merge(cust);
    }

    @Override
    public void deleteCustomer(int cid) {
        Customer cust =entityManager.find(Customer.class, cid);
        if(cust!=null)
            entityManager.remove(cust);
    }
}
```




```
@Override
public Customer getCustomerByCid(int cid) {
// Customer cust = entityManager.getReference(Customer.class, cid);
// System.out.println("In CustomerDAOImpl : "+cust);
Customer cust = entityManager.find(Customer.class, cid);

return cust;
}

@Override
public List<Customer> getAllCustomers(){
String JPAQL = "from Customer cust";
Query query = entityManager.createQuery(JPAQL, Customer.class);
List<Customer> mylist = query.getResultList();
return mylist;
}

@Override
public List<Customer> getAllCustomers1(){

String SQL = "select * from mycustomers cust";
return entityManager.createNativeQuery(SQL, Customer.class).getResultList();

}

@Override
public List<Customer> getCustomersByCity(String city){

String JPAQL = "from Customer cust where cust.city=?1";
return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, city)
                .getResultList();

}

@Override
public Customer getCustomerByEmail(String email) {

String JPAQL = "from Customer cust where cust.email=?1";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .getSingleResult();

}
```



```
@Override
public List<Customer> getCustomersByPhone(int phone){
String JPAQL = "from Customer cust where cust.phone=?1";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, phone)
                .getResultList();
}

@Override
public List<Customer> getCustomersByEmailAndPhone(String email,int phone){
String JPAQL = "from Customer cust where cust.email=?1 and cust.phone=?2";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .setParameter(2, phone)
                .getResultList();
}

@Override
public List<Customer> getCustomersByEmailOrPhone(String email,int phone){
String JPAQL = "from Customer cust where cust.email=?1 or cust.phone=?2";

return entityManager.createQuery(JPAQL, Customer.class)
                .setParameter(1, email)
                .setParameter(2, phone)
                .getResultList();
}

@Override
public BigInteger getCustomerCount() {
String SQL = "select count(*) from mycustomers cust";

return (BigInteger) entityManager.createNativeQuery(SQL) .getSingleResult();
}

@Override
public BigInteger getCustomerCountByCity(String city) {
String SQL = "select count(*) from mycustomers cust where cust.city=?1";

return (BigInteger) entityManager.createNativeQuery(SQL)
                .setParameter(1, city)
                .getSingleResult();
}
```



```
@Override
public int getCustomerPhoneByEmail(String email) {
String SQL = "select phone from mycustomers cust where cust.email=?1";

return (Integer) entityManager.createNativeQuery(SQL)
.setParameter(1, email)
.getSingleResult();
}

@Override
public String getCustomerCityByEmail(String email) {
String SQL = "select city from mycustomers cust where cust.email=?1";

return (String) entityManager.createNativeQuery(SQL)
.setParameter(1, email)
.getSingleResult();
}

@Override
public List<Customer> getCustomers(String city,String email,int phone){
String JPAQL = "from Customer cust where cust.city=?1 and cust.email=?2 and
cust.phone=?3";

return entityManager.createQuery(JPAQL, Customer.class)
.setParameter(1,city)
.setParameter(2, email)
.setParameter(3, phone)
.getResultList();
}

@Override
public Map<String,Integer> getCustomersInfo(String city) {
String SQL = "select email,phone from mycustomers cust where cust.city=?1";

List mylist = entityManager.createNativeQuery(SQL)
.setParameter(1,city)
.getResultList();

Map<String,Integer> mymap= new HashMap<>();
for(Object obj:mylist) {
Object myobj [] = (Object [])obj;
mymap.put((String)myobj[0], (Integer)myobj[1]);
}

return mymap;
}
```



@Override

public List<Customer> getCustomersByNameContains(String name){

String JPAQL = "from Customer cust where cust.cname like :pattern";

return entityManager.createQuery(JPAQL, Customer.class)

.setParameter("pattern","%" + name + "%")

.getResultList();

}

}

5)application.properties

#Database Properties

spring.datasource.driver-class-name=com.mysql.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5

spring.datasource.username=root

spring.datasource.password=srinivas

#JPA Properties

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

#Log Properties

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %m %n



Questions

Q36) What are the Bean Dependencies required for SessionFactory?

Ans:

- 1) DataSource
- 2) HibernateProperties
- 3) Mapping Classes

Q37) What are the Bean Dependencies required for HibernateTemplate?

Ans:

SessionFactory

Q38) What are the Bean Dependencies required for HibernateTransactionManager?

Ans:

SessionFactory

Q39) What are the Bean Dependencies required for EntityManagerFactory?

Ans:

- 1) DataSource
- 2) HibernateProperties
- 3) Mapping Classes

Q40) What are the Bean Dependencies required for JpaTransactionManager?

Ans:

EntityManagerFactory

Q41) Is it required to configure EntityManagerFactory and JpaTransactionManager for using JPA with Spring Boot?

Ans:

- No Need to Configure EntityManagerFactory and JpaTransactionManager.
- These two will be auto-configured by Spring Boot.

Q42) What is the use of JpaVendorAdapter and what are the various JpaVendorAdapters available?

Ans:

- JpaVendorAdapter is used to specify the JPA provider
- Interface JpaVendorAdapter.
- SPI interface that allows to plug in vendor-specific behavior into Spring's EntityManagerFactory creators.
- Serves as single configuration point for all vendor-specific properties.
- JpaVendorAdapter implementation classes are
 1. HibernateJpaVendorAdapter
 2. EclipseLinkJpaVendorAdapter

Q43) What are the Beans which are AutoConfigured by Spring Boot from the following?

Ans:

You Can Cross check which of the following beans will be AutoConfigured by Spring Boot by Injecting as follows.

@Autowired(required = false) DataSource ds;	AutoConfigured
@Autowired(required = false) JdbcTemplate jtemp;	AutoConfigured
@Autowired(required = false) NamedParameterJdbcTemplate njtemp;	AutoConfigured
@Autowired(required = false) EntityManagerFactory emf;	AutoConfigured
@Autowired(required = false) JpaTransactionManager txManager;	AutoConfigured
@Autowired(required = false) SessionFactory sf;	No
@Autowired(required = false) HibernateTemplate htemp;	No
@Autowired(required = false) HibernateTransactionManager txManager;	No

Q44) How can I do the Persistence Operations with JPA?

Ans:

EntityManager can be used to perform Persistence Operations.

Q45) How Can I Inject the EntityManager in the required DAO?

Ans:

```
@PersistenceContext
EntityManager entityManager;
```



Q46) What is the Speciality of @PersistentContext annotation? How it works differently from @Autowired/@Resource?

Ans:

Q47) What is EntityManager?

Ans:

- Each EntityManager instance is associated with a persistence context.
- Within the persistence context, the entity instances and their lifecycle are managed.
- A persistence context is like a cache which contains a set of persistent entities
- When Transaction is finished, all persistent objects are detached from the EntityManager's persistence context and are no longer managed.

Q48) Who is the Provider of @PersistentContext ?

Ans: JPA

Q49) What are the IMP Methods of EntityManager?

Ans: Refer Notes – Page No: 62

Q50) When Connection Pool will be created in Spring Boot Application?

Ans: At the Application Start-UP

Q51) When Connection Pool will be created in Spring Application?

Ans: At the Application Start-UP

Q52) How to Insert the Record in JPA?

Ans:

```
entityManager.persist(cust);
```

Q53) How to Update the Record in JPA?

Ans:

```
entityManager.merge(cust);
```

Q54) How to Delete the Record in JPA?

Ans:

```
Customer cust=entityManager.find(Customer.class, cid);  
If(cust!=null)  
    entityManager.remove(cust);
```

Q55) How to Select the Record by Primary Key in JPA?

Ans:

Two Options :

- 1) Customer cust=entityManager.**getReference**(Customer.class, cid);
- 2) Customer cust=entityManager.**find**(Customer.class, cid);



Q56) What are the difference between getReference() and find() methods?

Ans:

Refer Video.

Q57) How to Select the Single Values from Database in JPA?

Ans:

Prefer to Use Native Queries

```
String SQL="select count(*) from mycustomers ";
```

```
BigInteger count= (BigInteger) entityManager.createNativeQuery(SQL).getSingleResult();
```